

C++ Arrays

In this tutorial, we will learn to work with arrays. We will learn to declare, initialize, and access array elements in C++ programming with the help of examples.

In C++, an array is a variable that can store multiple values of the same type.

C++ Array Declaration

```
dataType arrayName[arraySize];
```

For example,

```
int x[6];
```

Here,

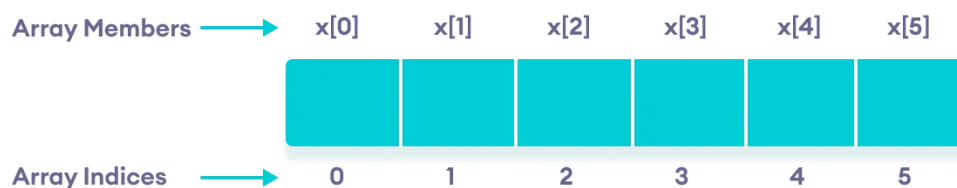
- `int` - type of element to be stored
 - `x` - name of the array
 - `6` - size of the array
-

Access Elements in C++ Array

In C++, each element in an array is associated with a number. The number is known as an array index. We can access elements of an array by using those indices.

```
// syntax to access array elements  
array[index];
```

Consider the array `x` we have seen above.



Elements of an array in C++

C++ Array Initialization

In C++, it's possible to initialize an array during declaration. For example,

```
// declare and initialize an array  
int x[6] = {19, 10, 8, 17, 9, 15};
```



C++ Array

elements and their data

Another method to initialize array during declaration:

```
// declare and initialize an array  
int x[] = {19, 10, 8, 17, 9, 15};
```

Here, we have not mentioned the size of the array. In such cases, the compiler automatically computes the size.

C++ Array With Empty Members

In C++, if an array has a size `n`, we can store upto `n` number of elements in the array. However, what will happen if we store less than `n` number of elements.

For example,

```
// store only 3 elements in the array
int x[6] = {19, 10, 8};
```

Here, the array `x` has a size of `6`. However, we have initialized it with only 3 elements.

In such cases, the compiler assigns random values to the remaining places.

Oftentimes, this random value is simply `0`.



Empty array

members are automatically assigned the value 0

How to insert and print array elements?

```
int mark[5] = {19, 10, 8, 17, 9}

// change 4th element to 9
mark[3] = 9;

// take input from the user
// store the value at third position
cin >> mark[2];

// take input from the user
// insert at ith position
cin >> mark[i-1];

// print first element of the array
cout << mark[0];

// print ith element of the array
cout >> mark[i-1];
```

Example 1: Displaying Array Elements

```
#include <iostream>
using namespace std;

int main() {
    int numbers[5] = {7, 5, 6, 12, 35};

    cout << "The numbers are: ";

    // Printing array elements
    // using range based for loop
    for (const int &n : numbers) {
        cout << n << " ";
    }
}
```

```

    cout << "\nThe numbers are: ";

    // Printing array elements
    // using traditional for loop
    for (int i = 0; i < 5; ++i) {
        cout << numbers[i] << " ";
    }

    return 0;
}

```

Output

```

The numbers are: 7 5 6 12 35
The numbers are: 7 5 6 12 35

```

Example 2: Take Inputs from User and Store Them in an Array

```

#include <iostream>
using namespace std;

int main() {
    int numbers[5];

    cout << "Enter 5 numbers: " << endl;

    // store input from user to array
    for (int i = 0; i < 5; ++i) {
        cin >> numbers[i];
    }

    cout << "The numbers are: ";

    // print array elements
    for (int n = 0; n < 5; ++n) {

```

```

        cout << numbers[n] << " ";
    }

    return 0;
}

```

Output

```

Enter 5 numbers:
11
12
13
14
15
The numbers are: 11  12  13  14  15

```

Example 3: Display Sum and Average of Array Elements Using for Loop

```

#include <iostream>
using namespace std;

int main() {

    // initialize an array without specifying size
    double numbers[] = {7, 5, 6, 12, 35, 27};

    double sum = 0;
    double count = 0;
    double average;

    cout << "The numbers are: ";

    // print array elements
    // use of range-based for loop
    for (const double &n : numbers) {
        cout << n << " ";

        // calculate the sum
        sum += n;
    }
}

```

```
        // count the no. of array elements
        ++count;
    }

    // print the sum
    cout << "\nTheir Sum = " << sum << endl;

    // find the average
    average = sum / count;
    cout << "Their Average = " << average << endl;

    return 0;
}
```

Output

```
The numbers are: 7  5  6  12  35  27
Their Sum = 92
Their Average = 15.3333
```

Passing Array to a Function in C++ Programming

In this tutorial, we will learn how to pass a single-dimensional and multidimensional array as a function parameter in C++ with the help of examples.

In C++, we can pass arrays as an argument to a function. And, also we can return arrays from a function.

Syntax for Passing Arrays as Function Parameters

The syntax for passing an array to a function is:

```
returnType functionName(dataType arrayName[arraySize]) {  
    // code  
}
```

Let's see an example,

```
int total(int marks[5]) {  
    // code  
}
```

Here, we have passed an `int` type array named `marks` to the function `total()`. The size of the array is `5`.

Example 1: Passing One-dimensional Array to a Function

```
// C++ Program to display marks of 5 students  
  
#include <iostream>  
using namespace std;  
  
// declare function to display marks  
// take a 1d array as parameter  
void display(int m[5]) {  
    cout << "Displaying marks: " << endl;  
  
    // display array elements  
    for (int i = 0; i < 5; ++i) {  
        cout << "Student " << i + 1 << ": " << m[i] << endl;  
    }  
}
```

```
int main() {  
  
    // declare and initialize an array  
    int marks[5] = {88, 76, 90, 61, 69};  
  
    // call display function  
    // pass array as argument  
    display(marks);  
  
    return 0;  
}
```

Output

```
Displaying marks:  
Student 1: 88  
Student 2: 76  
Student 3: 90  
Student 4: 61  
Student 5: 69
```